

Project Progress Report: Floating-Point Dot Product Instruction Integration in PicoRV32 Core

Rajdeep Alapati
IMT2023592
IIIT Bangalore
Alapati.Rajdeep@iitb.ac.in

Vinay Kusumanchi
IMT2023608
IIIT Bangalore
Kusumanchi.Vinay@iitb.ac.in

Abstract

This report outlines the week-wise progress for our Project Elective (PE), focusing on the integration of a custom floating-point Dot Product instruction within the PicoRV32 RISC-V core. The project spans from understanding the base repository and IEEE 754 standard to overcoming instruction encoding limitations and finally synthesizing a 16-pair fused dot-product architecture in Xilinx Vivado.

Week-Wise Progress Summary

Week 1: Project Initiation & Task Allocation

- Held our first project meeting with Professor Subir to define the scope of our Project Elective (PE). Received the primary topic and objective: extending the RISC-V instruction set by integrating a custom arithmetic instruction into the PicoRV32 core pipeline.

Week 2–3: Codebase Comprehension & Arithmetic Foundations

- Conducted a detailed walkthrough of the PicoRV32 processor flow using the Yosys GitHub repository. Also, we studied the IEEE 754 documentation to understand single-precision floating-point representation and its requirements for hardware implementation.

Week 4: Baseline Integer Instruction Integration

- To gain a practical understanding of the processor's control flow and decoding logic, we successfully added custom, generic integer-based instructions (Addition, Subtraction, and Multiplication) using behavioral modeling.

Week 5: Structural IEEE 754 Implementation

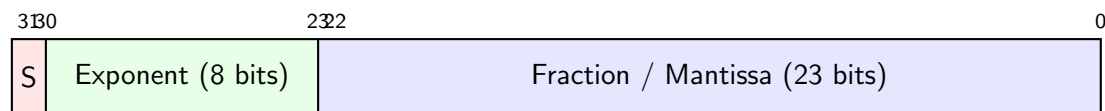
- Leveraging the understanding gained from the behavioral integration, we advanced to modifying the core to support a structural implementation of an IEEE 754 floating-point-based instruction.

IEEE 754 Single-Precision Floating-Point Format:

The structural logic implemented was based on the standard 32-bit floating-point equation:

$$Value = (-1)^S \times 2^{(E-Bias)} \times (1 + F)$$

Where the Bias for single precision is 127. The bits are allocated as follows:



Week 6: Dot Product Architecture Analysis

- Shifted focus to a novel Dot Product architecture, analyzing a reference research paper provided by our project advisor Professor Subir to understand the underlying datapath and logic.

Week 7–8: Debugging & Verification

- Discovered several errors in the baseline implementation code referenced from the paper. Dedicated this phase to debugging the logic, fixing the architectural flaws, and successfully achieving a working, functional model.

Mid-Semester Break

- Mid-semester exam preparation and scheduled academic break.

Week 12: FMA Synthesis & Instruction Constraint Resolution

- Proceeded to synthesize the Fused Multiply-Add (FMA/Dot Product) unit in Vivado. Encountered an architectural limitation: standard 32-bit R-type instructions provide only two source register spaces, but the FMA operation inherently requires more parameters. We resolved this by repurposing the `funct7` field and hardwiring the fourth value to 1. This workaround allowed us to successfully compute $Y = A \times B + C$ within the 32-bit constraint.

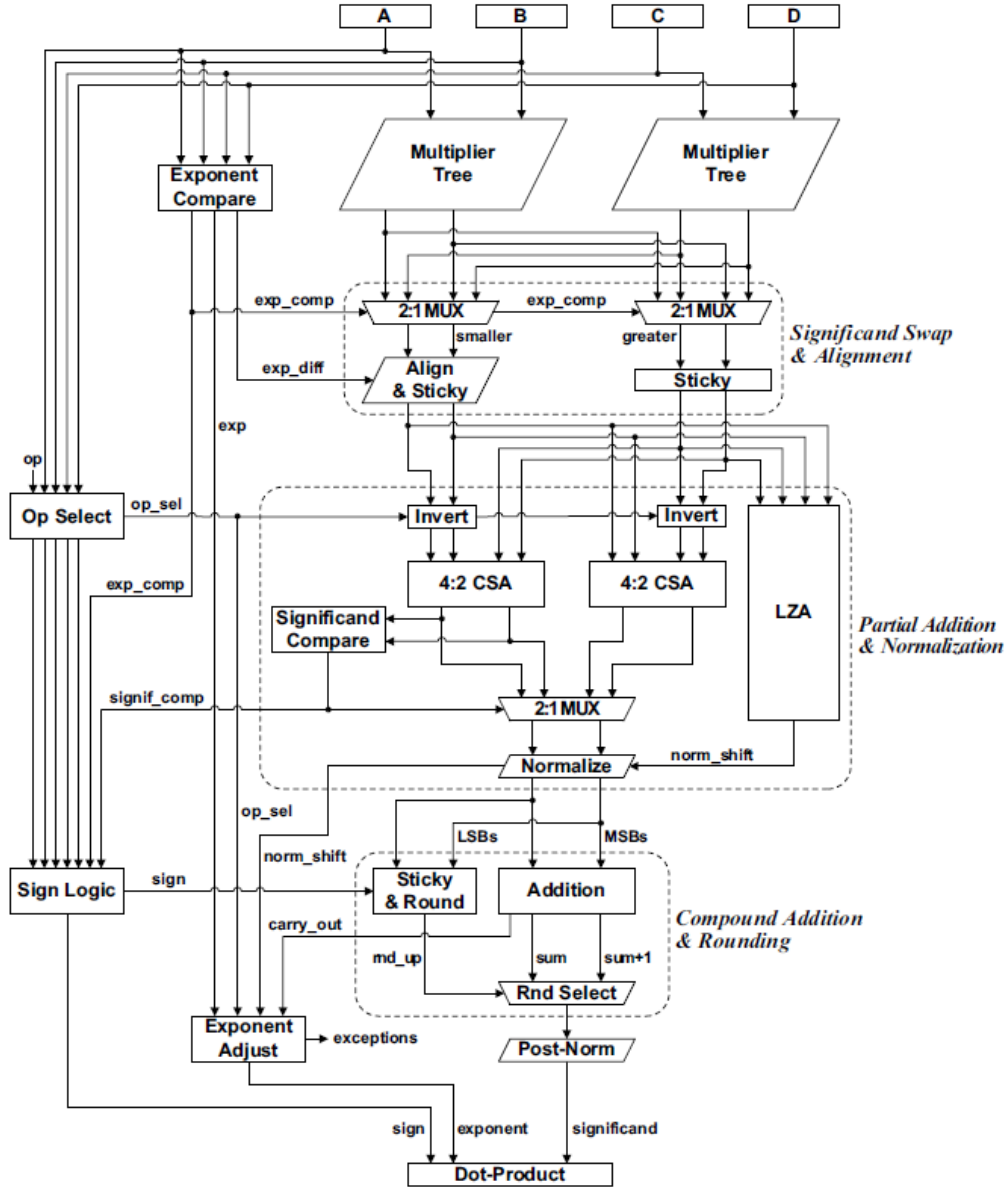


Figure 1: Hardware block diagram of the Fused Multiply-Add (FMA) / Dot Product architecture detailing the multiplier trees, significand alignment, and partial addition stages.

Week 13–14: Architecture Scaling (16-Pair Dot Product)

- Scaled the previously constrained architecture to support a 16-pair floating-point dot product instruction, effectively calculating $Y = A_1B_1 + A_2B_2 + A_3B_3 + \dots$ in hardware. Addressed the resulting logic depth and routing complexities.

Week 15: Final Submission Phase

- Compiled all codes and reports into final project documentation for end-of-semester submission.